

API Documentation

German Accident Data Platform — Unfallatlas Risk Lens

Base URL: <http://127.0.0.1:8000> | Docs: <http://127.0.0.1:8000/docs>

Overview

The platform exposes 15 REST endpoints serving data integrated from four official German open datasets. All endpoints use HTTP GET. Every response carries a provenance block listing sources and licences. An in-memory cache (300s TTL) is applied to all GET responses. Structured errors follow the shape: { error, status, detail, path }.

Endpoint Summary

#	Endpoint	Category	Description
1	GET /	Meta	Service index
2	GET /health	Meta	Liveness check
3	GET /regions	Regions	List regions by level or name
4	GET /regions/choropleth	GeoJSON	District polygons for choropleth maps
5	GET /regions/{ags}	Regions	Single region with indicator values
6	GET /accidents	List	Paginated accident rows
7	GET /accidents/near	Spatial	Accidents within radius of a point
8	GET /aggregates/accidents	Count	Total accident count for any filter
9	GET /aggregates/accidents/by-region	Ranking	Counts grouped by state/district
10	GET /aggregates/rate	Multi-source	Accidents per 100k cars/inhabitants
11	GET /aggregates/hotspots	Spatial	Severity-ranked crash clusters
12	GET /stats/first-year	Temporal	Earliest data year overall or per state
13	GET /stats/trend	Trend	Accident counts per year as time series
14	GET /metadata/sources	Provenance	Data sources and their licences
15	GET /import-runs	Provenance	Import audit trail with timestamps

Error Response Format

All errors return a consistent JSON object:

```
{ "error": true, "status": 404, "detail": "No region with AGS '99999'.",  
  "path": "/regions/99999" }
```

1. GET /

Service index — self-describing endpoint list

Returns the API name, version, and a sorted list of all available endpoint paths.

Returns

JSON object with name, version, docs URL, and endpoints array.

HTTP Status Codes

Code	Meaning
200	OK — returns service index

Example Request

```
GET http://127.0.0.1:8000/
```

Example Response

```
{
  "name": "German Accident Data Platform",
  "version": "1.0.0",
  "docs": "/docs",
  "endpoints": ["/", "/health", "/regions", ...]
}
```

2. GET /health

Liveness check

Simple health check to confirm the API server is running.

Returns

JSON object: { "status": "ok" }

HTTP Status Codes

Code	Meaning
200	OK — server is running

Example Request

```
GET http://127.0.0.1:8000/health
```

Example Response

```
{ "status": "ok" }
```

3. GET /regions

List regions filtered by level or name

Returns a list of regions from the administrative hierarchy. Can be filtered by level (state/district/municipality) or by a name substring search.

Parameters

Parameter	Type	Required	Description
level	string	No	Filter by level: state district municipality
search	string	No	Case-insensitive name substring filter
limit	integer	No	Max results (default 100, max 5000)

Returns

Array of Region objects.

Response Schema

Field	Type	Description
ags	string	Official AGS region code (2/5/8 digits)
name	string null	Region name (null if not yet backfilled)
level	string	country state district municipality
parent_ags	string null	AGS of the parent region

HTTP Status Codes

Code	Meaning
200	OK — list of regions
400	Bad Request — invalid level value

Example Request

```
GET http://127.0.0.1:8000/regions?level=state
```

Example Response

```
[
  { "ags": "14", "name": "Sachsen", "level": "state", "parent_ags": "DG" },
  { "ags": "11", "name": "Berlin", "level": "state", "parent_ags": "DG" }
]
```

4. GET /regions/choropleth

District polygons shaded by value — GeoJSON for choropleth maps

Returns a GeoJSON FeatureCollection where each feature is a district polygon with an associated accident count or rate value. Powers the choropleth map in the frontend.

Parameters

Parameter	Type	Required	Description
metric	string	No	count (default) or rate10k
year	integer	Yes	Year for accident counts and indicator values

Returns

GeoJSON FeatureCollection with district polygons and accident values.

Response Schema

Field	Type	Description
type	string	Always FeatureCollection
metric	string	count or rate10k
value_label	string	Human-readable description of the value
year	integer	Year the values relate to
feature_count	integer	Number of district polygons returned
features[].geometry	GeoJSON	District polygon (Polygon or MultiPolygon)
features[].properties.ags	string	5-digit district AGS
features[].properties.name	string	District name
features[].properties.value	float null	Accident count or rate; null = no data

HTTP Status Codes

Code	Meaning
200	OK — GeoJSON FeatureCollection
404	No district geometries loaded — add districts.geojson and re-run import
400	Bad Request — invalid metric value

Example Request

```
GET http://127.0.0.1:8000/regions/choropleth?metric=count&year=2023
```

Example Response

```
{
  "type": "FeatureCollection",
  "metric": "count",
  "value_label": "accidents",
  "year": 2023,
  "feature_count": 400,
  "features": [
    {
      "type": "Feature",
      "geometry": { "type": "Polygon", "coordinates": [...] },
      "properties": { "ags": "14612", "name": "Dresden", "value": 2154 }
    }
  ]
}
```

5. GET /regions/{ags}

Single region with its indicator values

Returns a single region identified by its AGS code, along with all stored indicator values (cars, per-10k rates) for that region across all years.

Parameters

Parameter	Type	Required	Description
ags	string (path)	Yes	AGS code: 2-digit state, 5-digit district, or 8-digit municipality

Returns

Region object with nested indicators array.

HTTP Status Codes

Code	Meaning
200	OK — region with indicators
404	No region with that AGS

Example Request

```
GET http://127.0.0.1:8000/regions/14612
```

Example Response

```
{
  "ags": "14612", "name": "Dresden", "level": "district",
  "parent_ags": "14",
  "indicators": [
    { "code": "CARS", "name": "Registered passenger cars", "unit": "vehicles",
      "year": 2023, "value": 280000 }
  ]
}
```

6. GET /accidents

Filtered accident rows (paginated)

Returns individual accident records filtered by state, year, category, and participant type. Supports pagination via limit and offset.

Parameters

Parameter	Type	Required	Description
state	string	No	State code e.g. SN, BE, NW or 2-digit AGS
region	string	No	AGS prefix (2/5/8 digits)
year	integer	No	Filter by year (2016–2024)
month	integer	No	Filter by month (1–12)
category	integer	No	1=fatal, 2=serious, 3=light
pedestrian	boolean	No	Only pedestrian-involved accidents
bicycle	boolean	No	Only cyclist-involved accidents
car	boolean	No	Only car-involved accidents
motorcycle	boolean	No	Only motorcycle-involved accidents

limit	integer	No	Max rows (default 50, max 1000)
offset	integer	No	Pagination offset (default 0)

Returns

Paginated object with total count, items array, and provenance.

HTTP Status Codes

Code	Meaning
200	OK — paginated accident rows
400	Bad Request — invalid state code or parameter
422	Unprocessable Entity — parameter type/range error

Example Request

```
GET http://127.0.0.1:8000/accidents?state=SN&year=2023&category=1&limit=2
```

Example Response

```
{
  "total": 162, "limit": 2, "offset": 0,
  "items": [
    { "accident_id": 1, "municipality_ags": "14612000", "year": 2023,
      "category": 1, "is_pedestrian": 0, "is_bicycle": 1,
      "lon": 13.7373, "lat": 51.0504 }
  ],
  "provenance": [{ "source": "Unfallatlas", "license": "dl-de/by-2-0" }]
}
```


7. GET /accidents/near

Accidents within radius of a coordinate point (spatial query)

Returns all accidents within a given radius of a WGS84 coordinate. Uses a bounding-box SQL prefilter followed by exact haversine distance calculation. Results are sorted by distance ascending.

Parameters

Parameter	Type	Required	Description
lat	float	Yes	Latitude in WGS84 (47.0–55.5)
lon	float	Yes	Longitude in WGS84 (5.5–15.5)
radius_km	float	No	Search radius in km (default 2.0, max 50)
year	integer	No	Filter by year
category	integer	No	1=fatal, 2=serious, 3=light
limit	integer	No	Max results (default 500, max 5000)

Returns

Object with center coordinates, radius, total count, and items array sorted by distance.

HTTP Status Codes

Code	Meaning
200	OK — accidents near the point
400	Bad Request — coordinates out of Germany bounding box
422	Unprocessable Entity — parameter type error

Example Request

```
GET
http://127.0.0.1:8000/accidents/near?lat=51.05&lon=13.74&radius_km=1&year=2023
```

Example Response

```
{
  "center": { "lat": 51.05, "lon": 13.74 },
  "radius_km": 1,
  "total": 156,
  "items": [
    { "accident_id": 42, "municipality_ags": "14612000",
      "year": 2023, "category": 2, "distance_km": 0.023 }
```

```
],  
  "provenance": [...]  
}
```

8. GET /aggregates/accidents

Total accident count for any filter combination

Returns a single accident count for the given filter combination. This is the primary endpoint for answering mandatory examiner questions such as 'how many accidents in Saxony in 2023?'

Parameters

Parameter	Type	Required	Description
state	string	No	State code e.g. SN, BE, NW
region	string	No	AGS prefix
year	integer	No	Filter by year
month	integer	No	Filter by month (1–12)
category	integer	No	1=fatal, 2=serious, 3=light
pedestrian	boolean	No	Only pedestrian-involved accidents
bicycle	boolean	No	Only cyclist-involved accidents
car	boolean	No	Only car-involved accidents
motorcycle	boolean	No	Only motorcycle-involved accidents

Returns

CountResponse object with query echo, accident_count, and provenance.

HTTP Status Codes

Code	Meaning
200	OK — accident count
400	Bad Request — invalid state code

Example Request

```
GET http://127.0.0.1:8000/aggregates/accidents?state=SN&year=2023
```

Example Response

```
{
  "query": { "state": "SN", "year": 2023 },
  "accident_count": 12513,
  "provenance": [
    { "source": "Unfallatlas", "license": "dl-de/by-2-0",
      "license_url": "https://www.govdata.de/dl-de/by-2-0" }
  ]
}
```

9. GET /aggregates/accidents/by-region

Accident counts grouped by region level (rankings)

Groups accident counts by state, district, or municipality and returns a ranked list. Supports the same filters as /aggregates/accidents. Useful for 'top 10 districts' queries.

Parameters

Parameter	Type	Required	Description
level	string	No	state district municipality (default: state)
state	string	No	Limit ranking to a specific state
year	integer	No	Filter by year
category	integer	No	1=fatal, 2=serious, 3=light
pedestrian	boolean	No	Only pedestrian-involved accidents
bicycle	boolean	No	Only cyclist-involved accidents
order	string	No	desc (default) or asc
limit	integer	No	Max results (default 20, max 1000)

Returns

Array of AccidentCount objects ranked by accident_count.

HTTP Status Codes

Code	Meaning
200	OK — ranked region list
400	Bad Request — invalid level or state code

Example Request

```
GET
http://127.0.0.1:8000/aggregates/accidents/by-region?level=district&year=2023&limit=3
```

Example Response

```
[
  { "ags": "09162", "name": "München", "level": "district", "year": 2023,
    "accident_count": 4998 },
  { "ags": "05315", "name": "Köln", "level": "district", "year": 2023,
    "accident_count": 4805 },
  { "ags": "03241", "name": "Region Hannover", "level": "district", "year":
    2023, "accident_count": 4736 }
]
```

10. GET /aggregates/rate

Accidents per N of an indicator — multi-source cross-dataset query

Computes accidents per N of a regional indicator by joining Unfallatlas accident counts with GENESIS/Regionalstatistik indicator values on the AGS key. This is the mandatory multi-source endpoint. Returns null rate where the denominator is missing (graceful degradation). District-level denominators are automatically summed to state level via AGS prefix.

Parameters

Parameter	Type	Required	Description
indicator	string	No	CARS or ACC_PER_10K (default: CARS)
level	string	No	state district municipality (default: district)
year	integer	Yes	Year for both accidents and indicator values
per	integer	No	Scale factor (default: 100000)
category	integer	No	1=fatal, 2=serious, 3=light
order	string	No	desc=highest rate first (default), asc=lowest
limit	integer	No	Max results (default 20, max 1000)

Returns

Object with indicator metadata, results array, and provenance.

Response Schema

Field	Type	Description
indicator_name	string	Human-readable indicator name
unit	string	Unit description e.g. accidents per 100,000 vehicles
results[].ags	string	District AGS code
results[].name	string null	District name
results[].accident_count	integer	Total accidents in the region for the year
results[].denominator	float null	Indicator value (null if not loaded)
results[].per	integer	Scale factor (default 100000)
results[].rate	float null	Computed rate; null where denominator missing
provenance	array	Source attribution and licence information

HTTP Status Codes

Code	Meaning
200	OK — rate results (null rate where denominator missing)
404	Indicator not loaded — re-run import with the indicator file
400	Bad Request — invalid level

Example Request

```
GET
http://127.0.0.1:8000/aggregates/rate?indicator=CARS&level=district&year=2024&limit=3
```

Example Response

```
{
  "indicator_name": "Registered passenger cars (Pkw)",
  "unit": "accidents per 100,000 vehicles",
  "results": [
    { "ags": "01003", "name": "Lübeck", "level": "district", "year": 2024,
      "accident_count": 1118, "indicator": "CARS",
      "denominator": 102826, "per": 100000, "rate": 1087.27 },
    { "ags": "02101", "name": null, "rate": null }
  ],
  "provenance": [...]
}
```

11. GET /aggregates/hotspots

Severity-ranked crash clusters on a configurable grid

Snaps accidents onto a geographic grid and ranks cells by a severity-weighted score (fatal×10 + serious×3 + light×1). Returns the most dangerous locations by harm, not headcount. Grid precision controls the cell size.

Parameters

Parameter	Type	Required	Description
precision	integer	No	Grid resolution: 2≈1.1km, 3≈110m (default), 4≈11m
min_count	integer	No	Min crashes per cell to qualify (default 3)
year	integer	No	Filter by year
state	string	No	Limit to a state (e.g. SN, BE)

category	integer	No	1=fatal, 2=serious, 3=light
pedestrian	boolean	No	Only pedestrian-involved accidents
bicycle	boolean	No	Only cyclist-involved accidents
weighted	boolean	No	Rank by severity score (default true)
limit	integer	No	Max hotspots returned (default 20, max 500)

Returns

Object with severity weights, grid metadata, hotspot count, and hotspots array.

Response Schema

Field	Type	Description
hotspots[].lat	float	Cluster centroid latitude
hotspots[].lon	float	Cluster centroid longitude
hotspots[].district	string	District name
hotspots[].district_agc	string	5-digit district AGS
hotspots[].total	integer	Total crashes in the grid cell
hotspots[].fatal	integer	Fatal crashes in the cell
hotspots[].serious	integer	Serious injury crashes in the cell
hotspots[].light	integer	Light injury crashes in the cell
hotspots[].pedestrian	integer	Crashes involving pedestrians
hotspots[].bicycle	integer	Crashes involving cyclists
hotspots[].severity_score	integer	Weighted score: fatal×10 + serious×3 + light×1

HTTP Status Codes

Code	Meaning
200	OK — ranked hotspot list
400	Bad Request — invalid state code

Example Request

GET http://127.0.0.1:8000/aggregates/hotspots?state=SN&year=2023&limit=2

Example Response

```
{
  "ranked_by": "severity_score",
  "severity_weights": { "fatal": 10, "serious": 3, "light": 1 },
  "grid_precision": 3,
  "approx_cell_size_m": 111,
  "hotspot_count": 443,
  "hotspots": [
    { "lat": 51.434933, "lon": 14.248862,
      "district": "Bautzen", "district_ags": "14625",
      "total": 12, "fatal": 0, "serious": 5, "light": 7,
      "pedestrian": 2, "bicycle": 3, "severity_score": 22 }
  ],
  "provenance": [...]
}
```


12. GET /stats/first-year

Earliest available data year overall or per state

Returns the earliest year for which accident data exists. When a state code is provided, returns the first year for that state specifically. This directly answers the mandatory questions about NRW (2019) and Mecklenburg-Vorpommern (2020) joining the Unfallatlas rollout.

Parameters

Parameter	Type	Required	Description
state	string	No	State code e.g. NW, MV, SN — if omitted returns overall earliest year

Returns

Object with state (if filtered) and first_year integer.

HTTP Status Codes

Code	Meaning
200	OK — first year
400	Bad Request — unknown state code

Example Request

```
GET http://127.0.0.1:8000/stats/first-year?state=NW
```

Example Response

```
{ "state": "NW", "first_year": 2019 }
```

13. GET /stats/trend

Accident counts per year as a time series

Returns a year-by-year series of accident counts, optionally filtered by state and category. Covers all loaded years (2016–2024). Powers the trend line chart in the frontend.

Parameters

Parameter	Type	Required	Description
state	string	No	State code e.g. SN, BE — if omitted returns national trend
category	integer	No	1=fatal, 2=serious, 3=light

Returns

Object with state, series array of {year, accident_count}, and provenance.

HTTP Status Codes

Code	Meaning
200	OK — trend series
400	Bad Request — invalid state code

Example Request

```
GET http://127.0.0.1:8000/stats/trend?state=SN
```

Example Response

```
{
  "state": "SN",
  "series": [
    { "year": 2016, "accident_count": 13241 },
    { "year": 2017, "accident_count": 13105 },
    { "year": 2023, "accident_count": 12513 },
    { "year": 2024, "accident_count": 12890 }
  ],
  "provenance": [...]
}
```

14. GET /metadata/sources

All integrated data sources and their licences

Returns the list of all data sources loaded into the platform, including their names, URLs, and licence information. This provenance block is also embedded in every other endpoint response.

Returns

Array of Provenance objects.

HTTP Status Codes

Code	Meaning
200	OK — list of data sources

Example Request

```
GET http://127.0.0.1:8000/metadata/sources
```

Example Response

```
[
  { "source": "Unfallatlas",
    "license": "Datenlizenz Deutschland Namensnennung 2.0 (dl-de/by-2-0)",
    "license_url": "https://www.govdata.de/dl-de/by-2-0" },
  { "source": "GENESIS 46251 (registered cars)",
    "license": "Datenlizenz Deutschland Namensnennung 2.0 (dl-de/by-2-0)",
    "license_url": "https://www.govdata.de/dl-de/by-2-0" }
]
```

15. GET /import-runs

Full import audit trail with timestamps and row counts

Returns every import run recorded in the database, including which source was loaded, when it was retrieved, which file was used, and how many records were loaded. Satisfies the provenance and update-behaviour documentation requirement.

Returns

Array of import run objects.

HTTP Status Codes

Code	Meaning
200	OK — import run history

Example Request

```
GET http://127.0.0.1:8000/import-runs
```

Example Response

```
[
  { "import_run_id": 1, "source": "Unfallatlas",
    "retrieved_at": "2026-06-21T15:12:00+00:00",
    "snapshot": "Unfallorte_2016_LinRef.txt",
    "record_count": 151673 },
  { "import_run_id": 9, "source": "Unfallatlas",
    "retrieved_at": "2026-06-21T15:12:00+00:00",
    "snapshot": "Unfallorte2024_LinRef.csv",
    "record_count": 268519 }
]
```

Provenance and Update Behaviour

Data Source Mapping

Source	File	Retrieved	Mapping
Unfallatlas 2016–2024	Unfallorte{YEAR}_LinRef.{csv/txt}	Jun. 2026	Column-variant map → accidents table; AGS assembled from ULAND+UREGBEZ+UKREIS+UGEMEINDE
GENESIS 46251	registered_cars_2023_2024.csv	Jun. 2026	Col 1=AGS (5-digit), col 4=Pkw insgesamt → indicator_values, code=CARS, years 2020–2024
Regionalstatistik per-10k	accident_per_10000_per_city.csv	Jun. 2026	schluessel=AGS, wert=rate → indicator_values, code=ACC_PER_10K, year=2023
District boundaries	districts.geojson	Jun. 2026	RS property (first 5 digits = AGS) → regions.geometry as GeoJSON text

Update Behaviour

The entire database is rebuilt from scratch on every import run: `python -m etl.run_import`

There is no incremental update. The database file is deleted and re-initialised, all four sources are loaded in order, and every load is recorded in `import_runs` with a timestamp and row count. To add a new year of Unfallatlas data, place the CSV in `data/raw/unfallatlas/{year}/` and re-run the import script. No code changes are required.

Licence

All integrated sources are published under Datenlizenz Deutschland – Namensnennung 2.0 (dl-de/by-2-0).

Licence URL: <https://www.govdata.de/dl-de/by-2-0>

Every API response includes a provenance block with source name, licence name, and licence URL. Clients are not required to track attribution separately.